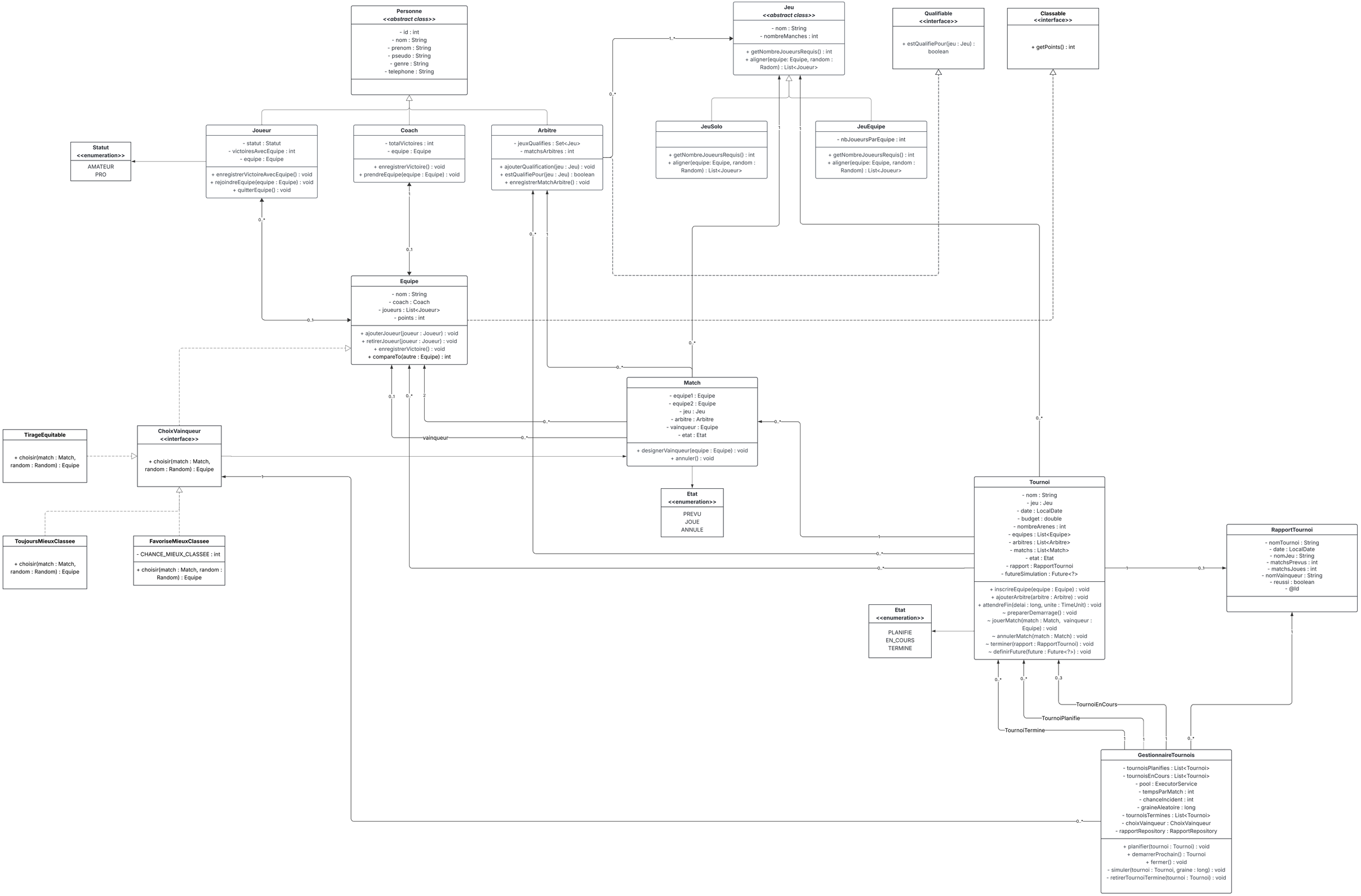

Personne
<<abstract class>>
- id : int
- nom : String
- prenom : String
- pseudo : String
- genre : String
- telephone : String
Jeu
<<abstract class>>
- nom : String
- nombreManches : int
+ getNombreJoueursRequis() : int
+ aligner(equipe: Equipe, random : Radom) : List<Joueur>
Qualifiable
<<interface>>
+ estQualifiePour(jeu : Jeu) : boolean
Classable
<<interface>>
+ getPoints() : int
Statut
<<enumeration>>
AMATEUR
PRO
Joueur
- statut : Statut
- victoiresAvecEquipe : int
- equipe : Equipe
+ enregistrerVictoireAvecEquipe() : void
+ rejoindreEquipe(equipe : Equipe) : void
+ quitterEquipe() : void
Coach
- totalVictoires : int
- equipe : Equipe
+ enregistrerVictoire() : void
+ prendreEquipe(equipe : Equipe) : void
Arbitre
- jeuxQualifies : Set<Jeu>
- matchsArbitres : int
+ ajouterQualification(jeu : Jeu) : void
+ estQualifiePour(jeu : Jeu) : boolean
+ enregistrerMatchArbitre() : void
JeuSolo
+ getNombreJoueursRequis() : int
+ aligner(equipe: Equipe, random : Random) : List<Joueur>
JeuEquipe
- nbJoueursParEquipe : int
+ getNombreJoueursRequis() : int
+ aligner(equipe: Equipe, random : Random) : List<Joueur>
Equipe
- nom : String
- coach : Coach
- joueurs : List<Joueur>
- points : int
+ ajouterJoueur(joueur : Joueur) : void
+ retirerJoueur(joueur : Joueur) : void
+ enregistrerVictoire() : void
+ compareTo(autre : Equipe) : int
Match
- equipe1 : Equipe
- equipe2 : Equipe
- jeu : Jeu
- arbitre : Arbitre
- vainqueur : Equipe
- etat : Etat
+ designerVainqueur(equipe : Equipe) : void
+ annuler() : void
Etat
<<enumeration>>
PREVU
JOUE
ANNULE
TirageEquitable
+ choisir(match : Match, random : Random) : Equipe
ChoixVainqueur
<<interface>>
+ choisir(match : Match, random : Random) : Equipe
ToujoursMieuxClassee
+ choisir(match : Match, random : Random) : Equipe
FavoriseMieuxClassee
- CHANCE_MIEUX_CLASSEE : int
+ choisir(match : Match, random : Random) : Equipe
Tournoi
- nom : String
- jeu : Jeu
- date : LocalDate
- budget : double
- nombreArenes : int
- equipes : List<Equipe>
- arbitres : List<Arbitre>
- matchs : List<Match>
- etat : Etat
- rapport : RapportTournoi
- futureSimulation : Future<?>
+ inscrireEquipe(equipe : Equipe) : void
+ ajouterArbitre(arbitre : Arbitre) : void
+ attendreFin(delai : long, unite : TimeUnit) : void
~ preparerDemarrage() : void
~ jouerMatch(match : Match, vainqueur : Equipe) : void
~ annulerMatch(match : Match) : void
~ terminer(rapport : RapportTournoi) : void
~ definirFuture(future : Future<?>) : void
Etat
<<enumeration>>
PLANIFIE
EN_COURS
TERMINE
RapportTournoi
- nomTournoi : String
- date : LocalDate
- nomJeu : String
- matchsPrevus : int
- matchsJoues : int
- nomVainqueur : String
- reussi : boolean
- @Id
GestionnaireTournois
- tournoisPlanifies : List<Tournoi>
- tournoisEnCours : List<Tournoi>
- pool : ExecutorService
- tempsParMatch : int
- chanceIncident : int
- graineAleatoire : long
- tournoisTermines : List<Tournoi>
- choixVainqueur : ChoixVainqueur
- rapportRepository : RapportRepository
+ planifier(tournoi : Tournoi) : void
+ demarrerProchain() : Tournoi
+ fermer() : void
- simuler(tournoi : Tournoi, graine : long) : void
- retirerTournoiTermine(tournoi : Tournoi) : void
vainqueur
TournoiEnCours
TournoiPlanifie
TournoiTermine